ONEDRAFT

CAD-AI — Aria Powered

Executable Application Stack & Service Architecture

Version 0.1

Status: Implementation Specification
Architecture: Modular Service-Oriented Monorepo
Primary Runtime: Python
API Runtime: FastAPI
Database: PostgreSQL
Cache / Queue: Redis
Object Storage: S3-Compatible Storage
Frontend: TypeScript / React
Workers: Python
API Version: `/api/v1`
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered

---

1. APPLICATION STACK PRINCIPLE

OneDraft shall be implemented as a computational application rather than as a conventional document-generation application.

The application stack shall preserve a strict separation between:

API

Domain Logic

Project Model

Geometry

Compliance

Validation

Documentation

AI Orchestration

Persistence

Background Processing

Object Storage

Frontend

The architecture shall ensure that no presentation-layer component becomes the authoritative source of project state.

The authoritative project state remains the versioned Project Model stored through the domain and persistence layers.

The application therefore follows:

```
CLIENT
    ↓
API
    ↓
APPLICATION SERVICES
    ↓
DOMAIN
    ↓
PROJECT MODEL
    ↓
PERSISTENCE
```

with computational workers operating through controlled service boundaries:

```
DOMAIN
    ↓
JOB QUEUE
    ↓
WORKER
    ↓
COMPUTATION
    ↓
VALIDATION
    ↓
DOMAIN FINALIZATION
```

---

2. SYSTEM STACK

The initial OneDraft implementation shall consist of:

```
apps/
    web/
    api/
    worker/

packages/
    domain/
    application/
    geometry/
    compliance/
```

```
    validation/
    documentation/
    aria/
    persistence/
    contracts/
    events/
    storage/
    jobs/
    observability/

infra/
    postgres/
    redis/
    object-storage/
    reverse-proxy/

docs/
    openapi/
    architecture/
    specifications/

tests/
    unit/
    integration/
    contract/
    end-to-end/
```

This structure establishes the first executable software skeleton.

---

3. MONOREPO PRINCIPLE

OneDraft shall initially use a single repository.

The repository shall contain the frontend, API, workers, domain packages, computational packages, infrastructure configuration, tests and documentation.

The purpose is to maintain synchronized versions of:

```
DATABASE
API
DOMAIN
WORKERS
FRONTEND
CONTRACTS
```

A single commit shall therefore be capable of representing a complete compatible OneDraft application state.

---

4. APPLICATION BOUNDARIES

The initial application shall be divided into the following logical services:

```
Web Application
API Service
Project Model Service
Aria Service
Geometry Service
Compliance Service
Validation Service
Documentation Service
Job Service
Worker Runtime
Persistence Service
Object Storage Service
Event Service
```

These are initially implemented as modular packages and processes rather than independently deployed microservices.

The architecture must permit later extraction into independently deployed services without requiring redesign of the domain model.

---

5. API SERVICE

The API service shall be responsible for:

```
Authentication
Authorization
Request validation
Response serialization
Idempotency
Project routing
Command submission
Job submission
Event publication
Error normalization
API versioning
```

The API service shall not contain computational geometry algorithms or direct AI-provider logic.

The API service communicates with application services.

---

6. API RUNTIME

The initial API runtime shall use:

```
Python
```

```
FastAPI
Pydantic
Uvicorn
```

The API shall expose:

```
/api/v1
```

OpenAPI shall be generated from the API implementation and maintained against:

```
/docs/openapi/openapi.yaml
```

The checked-in OpenAPI contract remains the machine-readable interface contract.

---

7. DOMAIN PACKAGE

The domain package is the central business-rule layer.

It shall contain:

```
Project
Requirement
Site
Building
Level
Grid
Space
Element
Assembly
Constraint
Relationship
ModelVersion
Revision
Command
Redline
View
Drawing
Sheet
CodeManifest
ValidationResult
DocumentPackage
Acceptance
```

The domain package shall not depend on:

```
FastAPI
React
PostgreSQL drivers
Redis
specific AI providers
specific object-storage providers
```

The domain must remain executable independently of infrastructure.

---

8. APPLICATION SERVICE LAYER

Application services orchestrate domain operations.

Initial services:

```
ProjectService
RequirementService
ModelService
CommandService
RevisionService
RedlineService
DrawingService
ComplianceService
ValidationService
DocumentService
AcceptanceService
```

Each service shall coordinate:

```
authorization
↓
domain operation
↓
persistence
↓
events
↓
jobs
```

where applicable.

---

9. PROJECT MODEL SERVICE

The Project Model Service is the primary mutation boundary.

No external client shall directly mutate model tables.

The mutation flow shall be:

```
REQUEST
 ↓
AUTHORIZATION
 ↓
LOAD CURRENT MODEL
 ↓
VERIFY EXPECTED REVISION
 ↓
LOAD CONSTRAINTS
 ↓
EXECUTE DOMAIN COMMAND
```

```
 ↓
VALIDATE MODEL TRANSITION
 ↓
CREATE NEW MODEL VERSION
 ↓
CREATE REVISION
 ↓
RECORD CHANGE RECORDS
 ↓
PERSIST
 ↓
EMIT EVENT
```

The resulting model version becomes the authoritative state only after successful transaction completion.

---

10. COMMAND MODEL

All meaningful project mutations shall be represented as commands.

Examples:

```
CreateProject
CreateBuilding
CreateLevel
CreateSpace
CreateElement
MoveElement
ResizeElement
DeleteElement
ChangeAssembly
AddConstraint
RemoveConstraint
CreateView
CreateDrawing
CreateRedline
ResolveRedline
GenerateDocuments
RunValidation
```

Commands shall be explicit and serializable.

A command shall contain:

```
command_id
project_id
expected_revision
operation
targets
parameters
preserved_constraints
actor
metadata
```

---

11. COMMAND EXECUTION

Command execution shall follow:

```
COMMAND
 ↓
AUTHORIZATION
 ↓
CURRENT REVISION CHECK
 ↓
TARGET RESOLUTION
 ↓
CONSTRAINT RESOLUTION
 ↓
DOMAIN MUTATION
 ↓
MODEL VALIDATION
 ↓
PERSISTENCE TRANSACTION
 ↓
REVISION CREATION
 ↓
EVENT
```

A command must never silently modify a newer revision than the one against which it was submitted.

---

12. OPTIMISTIC CONCURRENCY

OneDraft shall use optimistic concurrency control.

Every mutation shall identify an expected revision.

Example:

```
expected_revision = 14
```

If the current project revision is:

```
15
```

the command shall not be automatically applied.

The system shall return a revision conflict.

This protects against:

```
simultaneous users
simultaneous Aria operations
stale browser state
stale workers
duplicate requests
```

---

13. TRANSACTION SERVICE

The persistence layer shall expose transaction boundaries to the application layer.

Conceptually:

```
with unit_of_work() as uow:

    command = load_command()

    model = uow.projects.load(project_id)

    result = command.execute(model)

    uow.revisions.create(result.revision)
    uow.changes.record(result.changes)

    uow.commit()
```

The implementation may use different internal mechanics, but the transaction boundary must remain explicit.

---

14. UNIT OF WORK

The initial persistence abstraction shall provide:

```
ProjectRepository
RequirementRepository
BuildingRepository
LevelRepository
SpaceRepository
ElementRepository
ConstraintRepository
RelationshipRepository
RevisionRepository
CommandRepository
DrawingRepository
RedlineRepository
ComplianceRepository
ValidationRepository
DocumentRepository
AcceptanceRepository
EventRepository
```

The application layer shall depend on repository interfaces rather than direct SQL statements.

---

15. REPOSITORY PRINCIPLE

Repositories shall handle persistence.

Domain services shall handle business logic.

Workers shall handle expensive computation.

The API shall handle external communication.

The frontend shall handle presentation and interaction.

These concerns shall not be merged.

---

16. REDIS

Redis shall initially provide:

```
job queue coordination
short-lived cache
distributed locks where required
job progress
rate limiting support
temporary computation state
```

Redis shall not become the authoritative project database.

Project truth remains PostgreSQL.

---

17. JOB QUEUE

Expensive operations shall be submitted to the job system.

Initial job types:

```
GEOMETRY_GENERATION
GEOMETRY_VALIDATION
ARIA_PROCESSING
CODE_ANALYSIS
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
EXPORT
HASHING
```

Each job shall contain:

```
job_id
project_id
revision_id
model_version_id
job_type
payload
status
attempt
```

```
created_at
started_at
completed_at
error
```

---

18. JOB STATE MACHINE

Jobs shall use:

```
QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
```

Invalid transitions shall be rejected.

For example:

```
COMPLETED → RUNNING
```

is not permitted.

---

19. JOB IDEMPOTENCY

A job shall be uniquely associated with the project state for which it was generated.

Where appropriate, the system shall compute:

```
job_fingerprint
```

from:

```
project_id
model_version_id
revision_id
operation
parameters
```

Identical jobs may then be deduplicated.

---

20. WORKER SERVICE

The worker service shall execute asynchronous operations.

Initial worker modules:

```
workers/
    aria_worker.py
    geometry_worker.py
    compliance_worker.py
    validation_worker.py
    drawing_worker.py
    document_worker.py
```

Workers shall not directly mutate arbitrary project records.

They shall communicate through application services and controlled persistence interfaces.

---

21. WORKER VERSIONING

Every computational worker shall identify its implementation version.

Example:

```
geometry_engine_version
drawing_engine_version
validation_engine_version
document_engine_version
aria_orchestrator_version
```

The generated artifact metadata shall retain these versions.

This enables reproducibility.

---

22. STALE JOB PROTECTION

Before committing worker results, the worker must verify:

```
project_id
revision_id
model_version_id
```

against current project state.

If the target model version is no longer valid, the result shall not become current project state.

The job shall be marked:

```
INVALIDATED
```

or:

```
CANCELLED
```

according to the job lifecycle.

---

23. OBJECT STORAGE

Large binary artifacts shall not be stored directly in PostgreSQL.

Object storage shall contain:

```
geometry artifacts
drawing exports
PDF files
document packages
source documents
rendered previews
generated images
code-source artifacts
temporary computation artifacts
```

PostgreSQL stores references and metadata.

---

24. OBJECT STORAGE STRUCTURE

The logical object-storage namespace shall be:

```
projects/
    {project_id}/
        models/
        geometry/
        drawings/
        documents/
        compliance/
        source/
        previews/
        exports/
```

Objects shall be immutable once attached to a finalized project revision.

---

25. ARTIFACT HASHING

Generated artifacts shall be hashed.

Initial hashing algorithm:

```
SHA-256
```

The hash shall be stored alongside the artifact reference.

Example:

```
document_hash
geometry_hash
source_hash
```

This provides artifact integrity verification.

---

26. MODEL SNAPSHOTS

The Project Model Service shall support reconstruction from:

```
model_version_id
```

A model snapshot may be materialized for computational workloads.

The snapshot shall contain:

```
project metadata
requirements
buildings
levels
spaces
elements
assemblies
constraints
relationships
```

The snapshot shall identify its originating model version.

---

27. MODEL SERIALIZATION

The model snapshot shall use a versioned machine-readable representation.

Initial representation:

```
JSON
```

The representation shall be deterministic.

Equivalent model states should produce equivalent canonical serialization.

This allows:

```
state hashing
change detection
job fingerprinting
reproducibility
```

---

28. CANONICAL MODEL HASH

A model version shall have:

`state_hash`

generated from canonical model serialization.

The hash provides a compact identity for the computational project state.

---

29. ARIA SERVICE

Aria shall operate as an orchestration layer over OneDraft capabilities.

The external conceptual interface remains:

`OneDraft → Aria`

Internally:

```
Aria Orchestrator
        ↓
Intent Parser
        ↓
Project Context Resolver
        ↓
Command Planner
        ↓
Constraint Resolver
        ↓
Command Executor
        ↓
Validation
        ↓
Revision
```

---

30. ARIA PROVIDER ADAPTER

The AI provider shall remain behind an adapter.

```
Aria
 ↓
AI Provider Interface
 ↓
Provider Adapter
 ↓
Model Provider
```

The OneDraft domain must not depend on a specific external model vendor.

---

31. ARIA TOOL BOUNDARY

Aria shall not receive unrestricted system access.

Aria may invoke explicitly registered capabilities.

Initial capabilities:

```
get_project
get_model
get_revision
get_constraints
get_requirements
get_code_context
propose_command
validate_command
execute_command
create_redline
run_validation
generate_drawings
generate_documents
```

---

32. ARIA COMMAND AUTHORIZATION

AI-generated commands shall be treated as proposals until they pass the applicable authorization and validation rules.

The lifecycle shall be:

```
AI INTENT
 ↓
PROPOSED COMMAND
 ↓
COMMAND VALIDATION
 ↓
AUTHORIZATION
 ↓
EXECUTION
 ↓
MODEL VALIDATION
 ↓
REVISION
```

---

33. ARIA CONFIDENCE

AI operations may carry a confidence value.

Confidence shall not override:

```
hard constraints
authorization
code validation
model integrity
revision control
```

A high-confidence AI proposal remains subject to the same domain controls as any other mutation.

---

34. GEOMETRY ENGINE

The geometry subsystem shall be independent from the API runtime.

Its responsibilities include:

```
2D geometry
3D geometry
topology
intersections
offsets
boolean operations
spatial relationships
bounding volumes
measurement
geometric validation
```

The geometry engine shall consume versioned model state and return versioned geometry artifacts.

---

35. GEOMETRY OBJECT MODEL

Geometry shall be associated with:

```
object_id
model_version_id
geometry_version
geometry_type
coordinate_system
bounds
storage_reference
geometry_hash
```

Geometry shall never be treated as an unversioned mutable file.

---

36. GEOMETRY COMPUTATION

The standard flow is:

```
MODEL VERSION
 ↓
GEOMETRY JOB
 ↓
GEOMETRY ENGINE
 ↓
GEOMETRY ARTIFACT
 ↓
GEOMETRY VALIDATION
 ↓
ARTIFACT REGISTRATION
```

---

37. COORDINATE SYSTEM

OneDraft shall maintain explicit coordinate-system metadata.

At minimum:

```
project coordinate system
building coordinate system
drawing coordinate system
sheet coordinate system
```

Transformations between coordinate systems must be explicit.

No drawing engine shall assume that world coordinates and sheet coordinates are interchangeable.

---

38. UNITS

The domain model shall retain explicit units where dimensional values are represented.

Initial supported systems:

```
METRIC
IMPERIAL
```

The internal computational representation should use a consistent canonical unit system.

Conversions shall occur at system boundaries.

---

39. COMPLIANCE ENGINE

The compliance subsystem shall operate against:

```
project requirements
project model
jurisdiction
```

```
code manifest
code rules
```

The workflow shall be:

```
PROJECT
 ↓
JURISDICTION
 ↓
CODE SOURCES
 ↓
CODE MANIFEST
 ↓
APPLICABLE RULES
 ↓
MODEL ANALYSIS
 ↓
VALIDATION RESULTS
```

---

40. COMPLIANCE ISOLATION

Regulatory analysis shall not be embedded directly inside the drawing generator.

The drawing generator consumes validated model information.

The compliance engine remains independently testable.

---

41. VALIDATION ENGINE

Validation shall operate across multiple domains.

Initial categories:

```
CODE
SPATIAL
ACCESSIBILITY
DOCUMENT
COORDINATION
GEOMETRY
MODEL_INTEGRITY
```

Each validation result shall identify:

```
rule
affected_objects
severity
status
description
source
revision
```

---

42. VALIDATION GATES

Final document generation shall require the applicable validation gates.

The system shall distinguish:

```
PASS
WARNING
FAIL
NOT_APPLICABLE
```

A warning does not automatically prevent generation.

A blocking failure shall prevent finalization where the applicable project policy requires it.

---

43. DRAWING ENGINE

The drawing engine converts the Project Model into documentation.

It shall generate:

```
plans
elevations
sections
details
schedules
diagrams
annotations
dimensions
sheet layouts
```

The drawing is derived from:

```
MODEL VERSION
+
VIEW
+
DRAWING RULES
+
SHEET TEMPLATE
```

---

44. VIEW GENERATION

Views shall be generated from explicit parameters:

```
view_type
orientation
```

```
section_plane
cut_depth
scale
visibility
discipline
level
```

Examples:

```
FLOOR_PLAN
REFLECTED_CEILING_PLAN
ROOF_PLAN
ELEVATION
BUILDING_SECTION
WALL_SECTION
DETAIL
```

---

45. DRAWING DETERMINISM

Given identical:

```
model_version
view_definition
drawing_rules
sheet_template
engine_version
```

the drawing engine should produce the same logical drawing result.

This is required for regression testing and provenance.

---

46. ARCH D SUPPORT

The initial sheet-template system shall support:

```
ARCH D
```

with explicit:

```
paper dimensions
orientation
title block
sheet number
project information
revision block
drawing area
graphic standards
```

Additional paper formats may be added without changing the drawing model.

---

47. DOCUMENTATION PIPELINE

The documentation pipeline shall be:

```
PROJECT MODEL
 ↓
VIEW GENERATION
 ↓
DRAWING GENERATION
 ↓
SHEET COMPOSITION
 ↓
SCHEDULE GENERATION
 ↓
DOCUMENT ASSEMBLY
 ↓
PDF / EXPORT
 ↓
HASH
 ↓
DOCUMENT PACKAGE
```

---

48. DOCUMENT PACKAGE

A document package shall reference:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
drawing_ids
sheet_ids
artifact references
document hash
generation metadata
```

The package therefore represents a reproducible project state.

---

49. FRONTEND APPLICATION

The web application shall use:

```
TypeScript
React
```

The frontend shall consume the generated API client.

It shall not duplicate backend business logic.

---

50. FRONTEND MODULES

Initial frontend modules:

```
Dashboard
Project Workspace
Project Specification
Model Explorer
Floor/Level Navigator
Drawing Viewer
Redline Workspace
Compliance Panel
Validation Panel
Document Package
Revision History
Acceptance
Account / Organization
```

---

51. PROJECT WORKSPACE

The Project Workspace becomes the primary OneDraft interface.

Conceptually:

```
PROJECT
 ├── SPECIFICATION
 ├── MODEL
 ├── DRAWINGS
 ├── REDLINES
 ├── COMPLIANCE
 ├── VALIDATION
 ├── DOCUMENTS
 └── HISTORY
```

---

52. MODEL EXPLORER

The model explorer shall expose the project hierarchy:

```
Project
 └── Building
      ├── Level
      │    ├── Space
      │    └── Elements
      ├── Level
      └── Level
```

Elements may expose:

```
parameters
assembly
host
relationships
constraints
geometry
revision history
```

---

53. DRAWING VIEWER

The drawing viewer shall support:

```
pan
zoom
sheet navigation
drawing selection
object selection
redline creation
revision comparison
annotation inspection
```

The viewer shall identify the model version represented by the displayed drawing.

---

54. REDLINE WORKSPACE

The redline workflow shall be:

```
DRAWING
 ↓
USER MARKUP
 ↓
REDLINE
 ↓
ARIA INTERPRETATION
 ↓
COMMAND PROPOSAL
 ↓
VALIDATION
 ↓
EXECUTION
 ↓
NEW REVISION
 ↓
REDRAW
 ↓
REDLINE RESOLUTION
```

---

55. API CLIENT GENERATION

The TypeScript frontend client shall be generated from:

```
docs/openapi/openapi.yaml
```

The frontend shall not manually redefine API request and response schemas where generated types are available.

---

56. CONTRACT TESTING

The API implementation and OpenAPI contract shall be tested for compatibility.

The system shall verify:

```
request schema
response schema
status codes
error schema
authentication requirements
pagination
```

Contract drift shall fail CI.

---

57. CONFIGURATION

Configuration shall be environment-based.

Development:

```
.env
```

Production:

```
environment / secret manager
```

The repository shall contain:

```
.env.example
```

No production secrets shall be committed to source control.

---

58. CORE ENVIRONMENT VARIABLES

Initial configuration includes:

```
DATABASE_URL
REDIS_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OBJECT_STORAGE_ACCESS_KEY
OBJECT_STORAGE_SECRET_KEY

API_BASE_URL

ARIA_PROVIDER
ARIA_MODEL

JWT_ISSUER
JWT_AUDIENCE

LOG_LEVEL
ENVIRONMENT
```

Provider-specific variables shall remain behind the AI adapter.

---

59. LOCAL DEVELOPMENT STACK

The initial development environment shall run:

```
Web
API
Worker
PostgreSQL
Redis
Object Storage
```

through:

```
docker-compose.yml
```

or equivalent container orchestration.

---

60. LOCAL DEVELOPMENT FLOW

A developer shall be able to execute:

```
clone repository
 ↓
install dependencies
 ↓
configure .env
 ↓
start infrastructure
 ↓
run migrations
 ↓
```

```
seed development database
 ↓
start API
 ↓
start worker
 ↓
start web application
```

and obtain a functional OneDraft development environment.

---

61. DATABASE MIGRATIONS

Migrations shall be version controlled.

The application shall never silently modify the production database schema at runtime.

Migration execution shall occur explicitly through the deployment process.

---

62. SEEDING

Development seed data shall create:

```
organization
user
test project
building
levels
spaces
basic architectural elements
assemblies
ARCH D template
development jurisdiction
```

Seed data shall never be confused with production regulatory data.

---

63. EVENT SYSTEM

Important domain actions shall generate events.

Initial events:

```
PROJECT_CREATED
REQUIREMENT_CREATED
MODEL_CREATED
MODEL_MODIFIED
REVISION_CREATED
COMMAND_PROPOSED
```

```
COMMAND_EXECUTED
REDLINE_CREATED
REDLINE_RESOLVED
VALIDATION_STARTED
VALIDATION_COMPLETED
DRAWINGS_GENERATED
DOCUMENT_PACKAGE_GENERATED
ACCEPTANCE_RECORDED
```

---

64. EVENT PAYLOAD

Each event shall identify:

```
event_id
event_type
project_id
actor_id
revision_id
object_id
timestamp
payload
schema_version
```

---

65. EVENT IMMUTABILITY

Events shall be append-only.

An event shall not be edited after publication.

If an incorrect event is emitted, a corrective event shall be generated.

---

66. OBSERVABILITY

The system shall provide:

```
structured logs
metrics
request tracing
job tracing
error tracking
health checks
```

Every API request shall receive:

```
request_id
```

The request ID shall propagate through asynchronous jobs where applicable.

---

67. HEALTH ENDPOINTS

The API shall expose:

```
GET /health
GET /ready
```

Health verifies process availability.

Readiness verifies required dependencies.

---

68. READINESS DEPENDENCIES

Readiness may verify:

```
PostgreSQL
Redis
Object Storage
```

External AI providers should not necessarily make the entire API unavailable.

Provider availability shall be evaluated by the Aria service.

---

69. ERROR HANDLING

All errors shall be normalized through the standard API error structure.

Internal exceptions shall not expose:

```
database credentials
stack traces
secret values
internal infrastructure details
```

Development environments may provide expanded diagnostics.

---

70. SECURITY BOUNDARY

The initial security architecture shall separate:

```
authentication
```

```
authorization
tenant isolation
project isolation
service authentication
secret management
artifact access
audit logging
```

---

71. TENANT ISOLATION

Every project query shall be scoped through:

```
organization_id
project_id
```

A user shall not gain access to another organization's project by supplying a different project UUID.

Authorization must occur before object retrieval where appropriate.

---

72. ARTIFACT ACCESS

Object-storage artifacts shall not be exposed as uncontrolled public objects.

The API shall issue controlled access to authorized users.

Where signed URLs are used, they shall have limited lifetime.

---

73. SERVICE-TO-SERVICE ACCESS

Workers shall authenticate when calling internal application services.

The architecture shall not assume that internal network location constitutes authorization.

---

74. CI PIPELINE

Every source-control change shall execute automated checks.

Initial pipeline:

```
FORMAT
  ↓
LINT
  ↓
```

```
TYPE CHECK
 ↓
UNIT TEST
 ↓
DATABASE TEST
 ↓
CONTRACT TEST
 ↓
INTEGRATION TEST
 ↓
BUILD
```

---

75. UNIT TESTING

Unit tests shall cover:

```
domain rules
command validation
constraint logic
revision logic
model transformations
authorization policies
serialization
```

The domain layer should be testable without PostgreSQL.

---

76. INTEGRATION TESTING

Integration tests shall verify:

```
API
 ↓
Application Service
 ↓
Domain
 ↓
PostgreSQL
```

and:

```
API
 ↓
Job Queue
 ↓
Worker
 ↓
Persistence
```

---

77. END-TO-END TEST

The first complete end-to-end test shall execute:

```
CREATE ORGANIZATION
 ↓
CREATE USER
 ↓
CREATE PROJECT
 ↓
CREATE REQUIREMENTS
 ↓
CREATE BUILDING
 ↓
CREATE LEVELS
 ↓
CREATE SPACES
 ↓
CREATE WALLS
 ↓
CREATE DOORS
 ↓
CREATE WINDOWS
 ↓
GENERATE MODEL GEOMETRY
 ↓
GENERATE PLANS
 ↓
GENERATE ELEVATIONS
 ↓
GENERATE SECTION
 ↓
GENERATE ARCH D SHEETS
 ↓
SUBMIT ARIA MODIFICATION
 ↓
VALIDATE COMMAND
 ↓
CREATE REVISION 2
 ↓
REGENERATE AFFECTED DRAWINGS
 ↓
CREATE REDLINE
 ↓
RESOLVE REDLINE
 ↓
RUN VALIDATION
 ↓
GENERATE DOCUMENT PACKAGE
 ↓
HASH PACKAGE
 ↓
RECORD ACCEPTANCE
 ↓
RECONSTRUCT PROJECT HISTORY
```

---

78. FAILURE TESTING

The system shall explicitly test:

```
stale revision
duplicate command
invalid geometry
constraint conflict
failed worker
cancelled job
missing code manifest
failed validation
missing drawing
document generation failure
unauthorized project access
invalid artifact
database transaction failure
```

The expected behavior shall be deterministic and recorded.

---

79. ROLLBACK PRINCIPLE

A failed model mutation shall never leave a partially committed project state.

Database transactions provide atomicity.

Computational workers shall use temporary artifacts until successful validation and registration.

Only validated artifacts become current references.

---

80. TEMPORARY ARTIFACTS

Worker output shall initially be stored as:

```
temporary
```

The artifact becomes authoritative only after:

```
computation
 ↓
validation
 ↓
version check
 ↓
registration
```

---

81. DOCUMENT FINALIZATION GATE

A final document package shall not be marked:

```
FINAL
```

until the required chain exists:

```
PROJECT
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION
 ↓
DRAWINGS
 ↓
DOCUMENT PACKAGE
```

---

82. ACCEPTANCE GATE

Customer acceptance shall reference the exact:

```
revision
model version
document package
code manifest
```

accepted.

Acceptance shall therefore never mean merely:

```
"latest files"
```

It shall mean:

```
specific reproducible project state
```

---

83. DOMAIN EVENT → JOB RELATIONSHIP

Events may trigger asynchronous work.

Example:

```
REVISION_CREATED
        ↓
```

```
GEOMETRY_RECALCULATION
        ↓
DRAWING_REGENERATION
        ↓
VALIDATION
```

The system shall not automatically regenerate every artifact when only a localized change is made unless required.

Affected-object analysis shall determine the computational scope.

---

84. IMPACT ANALYSIS

Every model mutation shall attempt to identify:

```
affected objects
affected relationships
affected views
affected drawings
affected schedules
affected validation rules
```

This allows OneDraft to regenerate only what has become stale.

---

85. DEPENDENCY GRAPH

The relationship table forms the initial project dependency graph.

The graph will eventually allow:

```
element
 ↓
space
 ↓
level
 ↓
view
 ↓
drawing
 ↓
sheet
 ↓
document package
```

and:

```
element
 ↓
constraint
 ↓
```

```
validation rule
 ↓
validation result
```

to be traversed computationally.

---

86. INCREMENTAL REGENERATION

OneDraft shall prefer incremental regeneration.

Example:

A wall movement may affect:

```
floor plan
elevation
building section
room area
door/window relationships
dimensions
schedules
code validation
```

but should not require unrelated drawings to be regenerated unnecessarily.

---

87. CACHE STRATEGY

Cacheable data may include:

```
project summaries
model snapshots
code-rule lookups
drawing previews
validation summaries
job status
```

Cached values must always carry enough version information to prevent stale data from being interpreted as current state.

---

88. NO CACHE AS SOURCE OF TRUTH

Redis and other caches shall never replace PostgreSQL as the authoritative transactional store.

Cache invalidation shall be tied to:

```
revision
```

```
model_version
object version
```

where applicable.

---

89. STORAGE ABSTRACTION

Object storage shall be exposed through:

```
ObjectStore
```

rather than direct provider-specific calls throughout the application.

Initial interface:

```
put()
get()
delete()
exists()
head()
signed_url()
```

---

90. QUEUE ABSTRACTION

Job execution shall be exposed through:

```
JobQueue
```

Initial interface:

```
enqueue()
cancel()
status()
retry()
```

The domain shall not depend directly on Redis APIs.

---

91. AI ABSTRACTION

AI access shall be exposed through:

```
AIProvider
```

with operations conceptually including:

```
generate()
```

```
structured_output()
classify()
extract()
```

The Aria orchestration layer remains responsible for converting AI output into valid OneDraft commands.

---

92. COMPUTATIONAL PROVIDER ABSTRACTION

The architecture shall similarly isolate:

```
GeometryEngine
DrawingEngine
DocumentEngine
ValidationEngine
```

This prevents the domain model from becoming coupled to one implementation.

---

93. PACKAGE DEPENDENCY RULE

The dependency direction shall remain:

```
contracts
    ↓
domain
    ↓
application
    ↓
infrastructure
    ↓
API / workers
```

Infrastructure shall depend on domain abstractions.

The domain shall not depend on infrastructure.

---

94. FORBIDDEN DEPENDENCIES

The following patterns are prohibited:

```
React → PostgreSQL
```

```
React → Redis
```

```
Domain → FastAPI

Domain → Redis

Domain → AI Provider

AI Provider → PostgreSQL

Drawing Engine → direct user authentication

Worker → unrestricted database mutation
```

---

95. API → DOMAIN RULE

The API translates external requests into application commands.

It does not implement architectural rules.

For example:

```
API:
"Move this wall 300 mm."

Domain:
"Can this wall move 300 mm without violating active constraints?"
```

The second question belongs to the domain and validation layers.

---

96. FRONTEND → API RULE

The frontend shall never assume that an operation succeeded because a request was submitted.

The frontend must consume the authoritative API response.

For asynchronous work:

```
SUBMIT
  ↓
JOB_ID
  ↓
POLL / EVENT
  ↓
STATUS
  ↓
RESULT
```

---

97. REAL-TIME UPDATES

The initial implementation may use polling for job status.

Future versions may add:

```
WebSocket
Server-Sent Events
```

without changing the underlying job model.

---

98. API RATE CONTROL

The API shall support rate limiting.

Rate policies may vary by:

```
organization
user
endpoint
job type
AI operation
```

Expensive operations shall receive stricter controls than ordinary reads.

---

99. AI COST CONTROL

Aria requests shall be metered independently from ordinary API requests.

Each AI operation may record:

```
request_id
project_id
command_id
provider
model
input metrics
output metrics
execution time
estimated cost
```

This information belongs to the operational/billing layer and shall not alter project truth.

---

100. BILLING SEPARATION

Billing shall remain separate from project-model logic.

A subscription or payment event shall never directly modify architectural project state.

Billing determines entitlement.

Authorization determines whether the user may execute an operation.

The domain determines whether the operation is valid.

---

101. ENTITLEMENT CHECK

Before expensive operations:

```
AUTHENTICATION
 ↓
ORGANIZATION MEMBERSHIP
 ↓
ENTITLEMENT
 ↓
AUTHORIZATION SCOPE
 ↓
DOMAIN VALIDATION
 ↓
EXECUTION
```

---

102. AUDIT REQUIREMENT

Every meaningful state mutation shall be traceable to:

```
actor
command
revision
project
timestamp
result
```

AI-generated operations shall additionally identify the originating Aria command.

---

103. REPRODUCIBILITY REQUIREMENT

A finalized project state shall identify the versions of:

```
OneDraft application
domain schema
geometry engine
drawing engine
validation engine
document engine
```

```
Aria orchestrator
AI provider adapter
code manifest
```

This permits future investigation of how a document package was produced.

---

104. SOFTWARE VERSION MANIFEST

Every finalized document package shall contain a software manifest conceptually containing:

```
onedraft_version
domain_version
geometry_version
drawing_version
validation_version
document_version
aria_version
api_version
```

---

105. DEVELOPMENT PHASES

The executable implementation shall proceed in the following order:

```
PHASE 1
Repository foundation

PHASE 2
PostgreSQL migration

PHASE 3
Domain types

PHASE 4
Persistence repositories

PHASE 5
Project Model Service

PHASE 6
FastAPI application

PHASE 7
Job infrastructure

PHASE 8
Aria command boundary

PHASE 9
Geometry subsystem

PHASE 10
```

```
Compliance subsystem

PHASE 11
Validation subsystem

PHASE 12
Drawing subsystem

PHASE 13
Document subsystem

PHASE 14
React frontend

PHASE 15
End-to-end integration
```

---

106. PHASE 1 REPOSITORY FOUNDATION

The first executable repository shall contain:

```
README.md
LICENSE
pyproject.toml
.env.example
docker-compose.yml

apps/
packages/
infra/
docs/
tests/
```

The repository must build before computational functionality is introduced.

---

107. PHASE 2 DATABASE FOUNDATION

The database package shall implement:

```
001_initial_schema.sql
```

or equivalent ordered migrations.

The migration shall establish:

```
schemas
tables
foreign keys
indexes
constraints
```

required by the v0.1 persistence specification.

---

108. PHASE 3 DOMAIN TYPES

The domain package shall establish:

```
Project
Building
Level
Space
Element
Constraint
Relationship
Revision
ModelVersion
Command
Redline
View
Drawing
Sheet
CodeManifest
ValidationRun
DocumentPackage
Acceptance
```

using typed Python models.

---

109. PHASE 4 PERSISTENCE

Repositories shall implement the database contract.

The first repository test shall verify:

```
create
read
update
version
delete/status
```

for the core project model.

---

110. PHASE 5 PROJECT MODEL SERVICE

The first executable domain operation shall be:

```
CREATE PROJECT
```

followed by:

```
CREATE BUILDING
CREATE LEVEL
CREATE SPACE
CREATE ELEMENT
```

The first mutation operation shall then be:

```
MOVE_ELEMENT
```

---

111. PHASE 6 API

The first functional API shall expose:

```
POST /api/v1/projects
GET /api/v1/projects/{project_id}

POST /api/v1/projects/{project_id}/buildings
POST /api/v1/projects/{project_id}/levels
POST /api/v1/projects/{project_id}/spaces
POST /api/v1/projects/{project_id}/elements

POST /api/v1/projects/{project_id}/aria/requests
POST /api/v1/projects/{project_id}/validation
POST /api/v1/projects/{project_id}/drawings/generate
POST /api/v1/projects/{project_id}/documents/generate

GET /api/v1/jobs/{job_id}
```

---

112. PHASE 7 JOB INFRASTRUCTURE

The first worker implementation shall support:

```
JOB CREATION
JOB QUEUE
JOB EXECUTION
JOB STATUS
JOB FAILURE
JOB RETRY
JOB COMPLETION
```

Before geometry and document computation are added, the queue itself must pass integration testing.

---

113. PHASE 8 ARIA FOUNDATION

Aria shall initially operate against a controlled tool registry.

The first tools shall be:

```
get_project
get_model
get_revision
get_constraints
propose_command
validate_command
execute_command
```

This establishes the AI-to-domain boundary before sophisticated architectural reasoning is added.

---

114. PHASE 9 GEOMETRY FOUNDATION

The first geometry implementation shall support:

```
points
lines
polylines
rectangles
polygons
bounding boxes
transforms
intersections
```

The purpose of the first implementation is not complete architectural geometry.

The purpose is to establish the versioned computational geometry pipeline.

---

115. PHASE 10 COMPLIANCE FOUNDATION

The first compliance implementation shall support:

```
code source
code rule
code manifest
applicability
validation rule
validation result
```

The architecture shall permit jurisdiction-specific rule sets to be added without changing the Project Model.

---

116. PHASE 11 VALIDATION FOUNDATION

The first validation rules shall include basic model integrity:

```
duplicate identifiers
invalid level relationships
invalid geometry references
missing required objects
constraint conflicts
revision consistency
```

Architectural and regulatory validation shall then expand from this foundation.

---

117. PHASE 12 DRAWING FOUNDATION

The first drawing engine shall generate:

```
floor plan
elevation
section
```

from the Project Model.

The first sheet composition shall support:

```
ARCH D
```

---

118. PHASE 13 DOCUMENT FOUNDATION

The first document engine shall assemble:

```
drawings
sheets
schedules
notes
```

into a versioned document package.

The package shall be hashed and registered.

---

119. PHASE 14 FRONTEND FOUNDATION

The first frontend shall allow a user to:

```
create project
enter requirements
```

```
view model
view levels
view spaces
view elements
submit Aria command
view command result
view revisions
view drawings
create redline
view validation
generate document package
```

---

120. PHASE 15 SYSTEM INTEGRATION

The complete stack shall then execute:

```
USER
 ↓
WEB
 ↓
API
 ↓
APPLICATION SERVICE
 ↓
DOMAIN
 ↓
POSTGRESQL
 ↓
EVENT
 ↓
QUEUE
 ↓
WORKER
 ↓
GEOMETRY / VALIDATION / DRAWING
 ↓
OBJECT STORAGE
 ↓
DOCUMENT PACKAGE
 ↓
WEB
 ↓
CUSTOMER ACCEPTANCE
```

---

121. FIRST STACK ACCEPTANCE TEST

The stack shall be considered operational when the system can execute the following without manual database intervention:

```
1. Start infrastructure.
```

2. Run database migrations.

3. Start API.

4. Start worker.

5. Start web application.

6. Authenticate a development user.

7. Create an organization.

8. Create a project.

9. Store project requirements.

10. Create a building.

11. Create levels.

12. Create spaces.

13. Create walls, doors and windows.

14. Generate a model version.

15. Submit an Aria command.

16. Validate the command.

17. Execute the command.

18. Create a new revision.

19. Queue geometry processing.

20. Generate drawing views.

21. Generate ARCH D sheets.

22. Create a redline.

23. Resolve the redline.

24. Run validation.

25. Generate a document package.

26. Hash the package.

27. Store the package.

28. Record acceptance.

29. Reconstruct the accepted project state.

30. Verify that the generated documentation corresponds exactly to the accepted revision.

---

122. FIRST PRODUCTION-READY BOUNDARY

The first production-ready boundary is not the complete architectural drawing engine.

It is the ability to reliably perform:

```
PROJECT
 →
MODEL
 →
COMMAND
 →
REVISION
 →
VALIDATION
 →
ARTIFACT
 →
PROVENANCE
```

Once this loop is stable, additional architectural intelligence can be added without destabilizing the underlying system.

---

123. ARCHITECTURAL INVARIANTS

The following invariants are mandatory:

```
PostgreSQL is the authoritative transactional store.

The Project Model is the authoritative architectural state.

Revisions are immutable historical states.

Model versions are immutable computational states.

Commands are the controlled mutation interface.

AI cannot directly mutate infrastructure.

Workers cannot bypass domain controls.

Geometry is versioned.

Drawings identify their source model version.

Documents identify their source revision.

Code manifests are versioned.
```

```
Validation results identify their source revision.

Final packages are hashed.

Audit events are append-only.
```

---

124. STACK STATUS

The OneDraft architecture now extends through:

```
PRODUCT DEFINITION
 ↓
SYSTEM ARCHITECTURE
 ↓
TECHNOLOGY ARCHITECTURE
 ↓
DOMAIN MODEL
 ↓
DATABASE SCHEMA
 ↓
OPENAPI CONTRACT
 ↓
APPLICATION SERVICE ARCHITECTURE
 ↓
PROJECT MODEL COMMAND BOUNDARY
 ↓
JOB / WORKER ARCHITECTURE
 ↓
ARIA ORCHESTRATION BOUNDARY
 ↓
GEOMETRY PIPELINE
 ↓
COMPLIANCE PIPELINE
 ↓
VALIDATION PIPELINE
 ↓
DRAWING PIPELINE
 ↓
DOCUMENT PIPELINE
 ↓
FRONTEND APPLICATION
 ↓
INTEGRATION TESTING
```

---

125. NEXT IMPLEMENTATION ARTIFACTS

The next executable engineering package shall now consist of:

Artifact A

`001_initial_schema.sql`

The executable PostgreSQL foundation.

Artifact B

`openapi.yaml`

The machine-readable API contract.

Artifact C

`domain_types.py`

The typed OneDraft domain model.

Artifact D

`project_model_service.py`

The transactional Project Model command boundary.

Artifact E

`repositories.py`

The persistence repository interfaces and initial PostgreSQL implementations.

Artifact F

`job_service.py`

The asynchronous job abstraction.

Artifact G

`worker.py`

The initial worker runtime.

Artifact H

`aria_service.py`

The controlled Aria orchestration boundary.

Artifact I

`docker-compose.yml`

The local executable infrastructure stack.

Artifact J

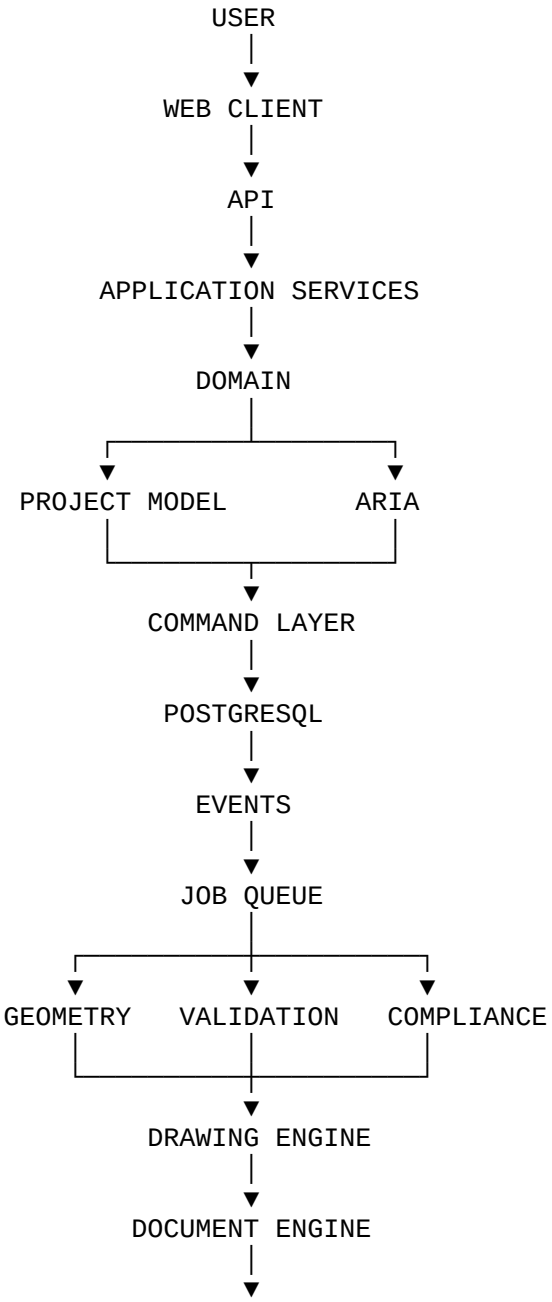
`tests/test_project_lifecycle.py`

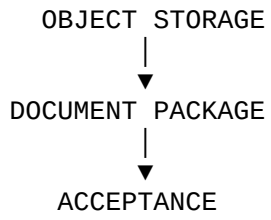
The first complete Project Model integration test.

---

126. FINAL ARCHITECTURAL DECISION

OneDraft shall be implemented as a layered computational platform:





The architectural drawing remains a derived artifact.

The document package remains a derived artifact.

The AI remains an orchestration and reasoning layer.

The Project Model remains the authoritative computational representation of the building.

---

127. SPECIFICATION STATUS

ONEDRAFT EXECUTABLE APPLICATION STACK & SERVICE ARCHITECTURE v0.1

STATUS: ESTABLISHED

The persistence/API specification has now been extended into the executable service stack.

The next engineering action is no longer another conceptual architecture document.

The next action is to create the actual repository artifacts:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
repositories.py
job_service.py
worker.py
aria_service.py
docker-compose.yml
tests/test_project_lifecycle.py
```

These artifacts constitute the first executable OneDraft software foundation.

The implementation sequence is therefore:

```
SCHEMA
 ↓
DOMAIN TYPES
 ↓
REPOSITORIES
 ↓
PROJECT MODEL SERVICE
 ↓
API
 ↓
JOB SYSTEM
 ↓
```

WORKER
 ↓
ARIA
 ↓
GEOMETRY
 ↓
VALIDATION
 ↓
DRAWINGS
 ↓
DOCUMENTS
 ↓
FRONTEND
 ↓
FULL INTEGRATION

 .